

OpenGL Outline Algorithms - Detailed Implementation Guide

Project: OpenGLOutline (Win32, OpenGL 3.3 compatibility)

1) Pipeline context

The scene (cube) writes color + depth/stencil into the main render target. Stencil writes value 1 for visible cube fragments. All outline implementations detect outside-edge pixels where center stencil is 0 but nearby stencil has at least one 1. The outline pass writes a composited texture, and the present pass displays it.

2) Shared edge condition

All modes reduce to the same binary edge test on outside pixels:

$\text{outsideEdge} = (\text{centerStencil} == 0) \text{ AND } (\text{exists sampled stencil texel} > 0).$

What changes across modes is how that 'exists sampled texel' condition is computed.

3) Brute-force mode (square neighborhood)

Shader path: `outline.frag` with `outlineImplementation == 0`.

Algorithm: For each pixel, sample all offsets (dx, dy) inside a square radius R around the pixel. If any sampled texel has stencil $\neq 0$ and center is 0, color as outline.

Detailed behavior

- Sampling footprint is a full $(2R+1) \times (2R+1)$ box. For $R=4$, that is 81 stencil fetches per pixel.
- The implementation clamps each sample coordinate to framebuffer bounds, so border pixels remain valid.
- Produces thick, robust silhouettes and fills diagonal gaps well.

Complexity

- Per pixel work is $O(R^2)$. Cost grows rapidly as thickness increases.
- Best suited to small-to-medium radii or when quality is preferred over speed.

Visual traits

- More isotropic than directional kernels, but still square/Manhattan biased compared to true Euclidean distance.

4) Cross-kernel mode (axis-only neighborhood)

Shader path: `outline.frag` with `outlineImplementation == 1`.

Algorithm: For each pixel, sample only two 1D rays: horizontal and vertical offsets in $[-R, R]$. No diagonal offsets are tested.

Detailed behavior

- Sampling footprint is $4R+1$ texels (center counted once). For $R=4$, that is 17 effective samples.
- Same center/outside decision as brute-force, but with sparse directional probes.

Complexity

- Per pixel work is $O(R)$. Much cheaper than brute-force for larger radii.

Visual traits

- Can look plus-shaped at larger thicknesses; diagonals may appear thinner or broken relative to brute-force.
- Useful when frame time is constrained and exactness is not critical.

5) Jump Flood Algorithm (JFA) mode

Shader paths: `jfa_seed.frag`, `jfa_step.frag`, `jfa_compose.frag`.

JFA converts outline detection into nearest-feature propagation, then thresholds Euclidean distance.

Pass A - Seed initialization

- For stencil==1 pixels: store their own pixel center coordinates as seed ($x+0.5, y+0.5$).
- For stencil==0 pixels: store invalid seed (-1, -1).

Pass B - Jump propagation

- Run multiple fullscreen passes with jump sizes $k = 2^n, 2^{(n-1)}, \dots, 1$.
- In each pass, examine 3×3 neighbors at spacing k, select candidate seed with minimum squared distance.
- Ping-pong between two RGBA32F textures so each pass reads prior state and writes next state.

Pass C - Compose

- For outside pixels, read nearest propagated seed and compute Euclidean distance to current pixel

center.

- If distance $\leq$ outlineWidth, mark outline; otherwise keep scene color.

Complexity

- Approximate per pixel work is $O(\log_2(\max(\text{width}, \text{height})) * 9)$, largely independent of outline thickness.
- Typically scales better than brute-force for thick outlines.

Visual traits

- Distance-based contour is smoother and closer to circular expansion than square/cross kernels.
- As an approximation algorithm, can have minor artifacts near sharp corners or very thin structures.

6) Thickness parameter behavior

- UI slider range is 1..32 pixels.
- Brute-force/Cross: thickness maps to integer sampling radius.
- JFA: thickness maps to Euclidean distance threshold during compose.
- Because meanings differ, equal numeric values may look slightly different between modes.

7) MSAA x8 interaction

- Scene pass can render into multisample color + depth/stencil textures.
- A resolve pass blits color/depth/stencil into single-sample textures before outline processing.
- This preserves stencil-driven logic while reducing base geometry aliasing.

8) Choosing a mode

- Use Brute-force for consistent thick silhouettes at moderate radii and simpler debugging.
- Use Cross-kernel for fastest non-JFA path when some directional bias is acceptable.
- Use JFA for large thickness values or when smoother distance-like expansion is desired.

9) References

- Khronos OpenGL Wiki - Framebuffer Object
https://www.khronos.org/opengl/wiki/Framebuffer_Object
- Khronos OpenGL Wiki - Multisampling
<https://www.khronos.org/opengl/wiki/Multisampling>
- Khronos OpenGL Wiki - Sampler (GLSL)
[https://www.khronos.org/opengl/wiki/Sampler_\(GLSL\)](https://www.khronos.org/opengl/wiki/Sampler_(GLSL))
- OpenGL glBlitFramebuffer reference
<https://registry.khronos.org/OpenGL-Refpages/gl4/html/glBlitFramebuffer.xhtml>
- Rong & Tan - Jump Flooding in GPU
<https://www.comp.nus.edu.sg/~tants/jfa.html>